

DualPipe vs. Pipeline Parallelism

GPipe $\rightarrow$ 1F1B $\rightarrow$ Zero-Bubble $\rightarrow$ DualPipe

Killing the pipeline bubble — and, for MoE, hiding the all-to-all behind compute

Prepared 2026-06-14 | systems comparison

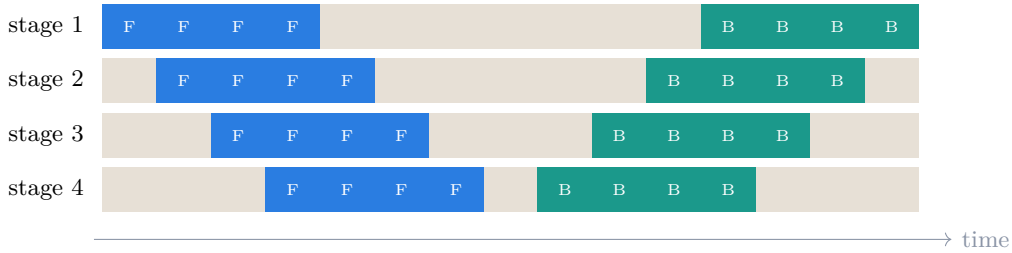
The problem. Pipeline parallelism splits a model’s layers across P devices (*stages*) and streams M micro-batches through them. But a stage can’t start until the one before it produces activations, so the pipeline takes time to *fill* and *drain* — during which most devices sit idle. That idle time is the **bubble**. Three things trade off: *bubble* (idle GPUs), *activation memory* (how many in-flight micro-batches you must store), and — for MoE — *communication* (the all-to-all that shuffles tokens to experts). DualPipe is the schedule that attacks all three.

legend: ■ forward F ■ backward B ■ bubble (idle) ■ all-to-all comm

1 GPipe — all forwards, then all backwards

the baseline

GPipe runs *all* M micro-batches forward, then *all* backward. Simple, but the fill+drain bubble is large and you must store activations for *all* M in-flight micro-batches.

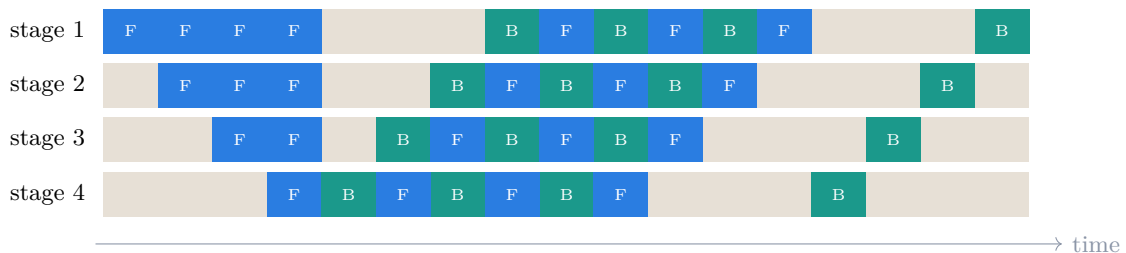


Bubble fraction $\approx \frac{P-1}{M}$ (fill + drain = $P-1$ slots out of $\sim M$). Shrinks only by using many micro-batches — but more micro-batches in GPipe means *more* stored activations. That tension is what every later method tries to break.

2 1F1B — interleave to cap memory

PipeDream-Flush / Megatron

One-forward-one-backward starts a backward as soon as a micro-batch finishes its first forward, so each stage alternates F and B in *steady state*. The bubble is the *same* $(P-1)(F+B)$, but you only ever store $\sim P$ activations (not M) — a big memory win that lets you raise M to shrink the bubble.



Interleaved 1F1B (Megatron) goes further: each device holds v *non-contiguous* layer chunks, cutting the bubble to $\frac{P-1}{vM}$ — at the cost of $v\times$ more point-to-point communication.

3 Zero-Bubble — split the backward

ZB-H1 / ZB-H2

The trick: a backward pass really does *two* jobs — compute the gradient w.r.t. the *input* (B , needed immediately by the previous stage) and w.r.t. the *weights* (W , needed only at the optimizer step). Split them:

$$\text{backward} = \underbrace{\quad\quad\quad}_B \quad + \quad \underbrace{\quad\quad\quad}_W .$$

grad to input — on the critical path grad to weights — deferrable

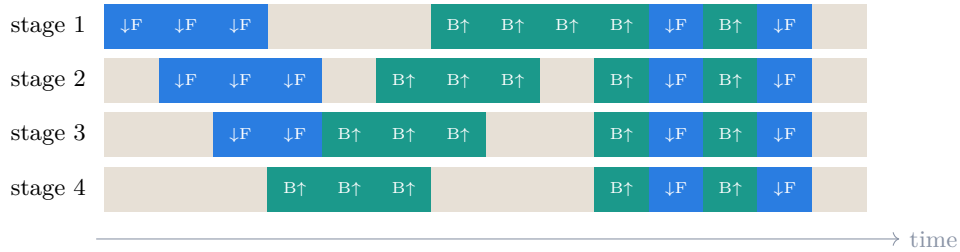
Because W has no downstream dependency, it can be *slotted into bubbles*. Scheduling B on the critical path and back-filling idle slots with W drives the bubble toward **zero** (ZB-H2). Memory like 1F1B; the cost is a more intricate schedule and bookkeeping for the deferred W .

4 DualPipe — bidirectional + overlap the comm

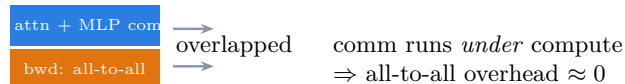
DeepSeek-V3

DualPipe adds *two* ideas on top of the B/W split, aimed squarely at large MoE training.

1. Bidirectional scheduling. Feed micro-batches in from *both ends* of the pipeline at once — two streams flowing in opposite directions. Each stream’s fill bubble is covered by the other stream’s work, so the two halves’ idle times overlap instead of adding:



2. Computation–communication overlap (the real prize). In expert-parallel MoE, each layer must *all-to-all dispatch* tokens to their experts and *all-to-all combine* the results — huge cross-node traffic. DualPipe splits every chunk into { attention, dispatch, MLP/experts, combine } and pairs a forward chunk with a backward chunk so that **one chunk’s compute hides the other chunk’s all-to-all**:



A fixed slice of SMs is reserved for the communication kernels so the overlap is reliable. The result: at scale the MoE all-to-all is *effectively free*, which is the single biggest cost in cross-node MoE training.

The price. Bidirectional scheduling needs the model parameters resident for *both* directions, so DualPipe holds **2×** the parameters. DeepSeek judged the trade worth it: parameter memory is cheap relative to the throughput won by erasing the bubble *and* the comm.

5 Side-by-side

Method	Bubble	Params	Activations	Idea
GPipe	$(P-1)(F+B)$	$1\times$	M	all- F then all- B
1F1B	$(P-1)(F+B)$	$1\times$	P	interleave F/B , cap memory
Interleaved 1F1B	$\frac{(P-1)(F+B)}{v}$	$1\times$	P	v chunks/device (+comm)
ZB1P	$(P-1)(F+B-2W)$	$1\times$	P	split B , backfill with W
DualPipe	$(\frac{P}{2}-1)(F\&B+B-3W)$	$2\times$	$P+1$	bidirectional + comm overlap

(F = forward time, B = input-grad backward, W = weight-grad backward, $F\&B$ = a mutually-overlapped forward/backward pair. Lower bubble is better.)

Bottom line. The lineage is one long campaign on the bubble: GPipe pays it in full; 1F1B keeps it but slashes memory; interleaving and Zero-Bubble whittle it down by chunking and by deferring weight gradients. **DualPipe** takes Zero-Bubble’s B/W split, adds a *bidirectional* schedule so two streams fill each other’s gaps, and — its defining move — *overlaps the MoE all-to-all with computation* so cross-node communication stops being a tax at all. The cost is $2\times$ parameter memory, which is exactly the kind of trade that pays off when the model is a giant sparse MoE and the network is the bottleneck. It is why V3 trained at its famous low cost.

GPipe: Huang et al. (2019). 1F1B/PipeDream: Narayanan et al. (2019); interleaved 1F1B: Megatron-LM. Zero Bubble: Qi et al. (2023). DualPipe and the bubble-formula table: DeepSeek-V3 Technical Report (arXiv:2412.19437); schedule drawings here are simplified for intuition, not exact slot-level reproductions.